The Domain Name System: A Four-Level Naming Hierarchy, Recursive and Iterative Resolution, Caching, and Cache Poisoning

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 The RFC 1035 Message Format **4**

3 System Architecture **6**

4 Implementation and Workflows **6**

4.1 Authoritative Servers 6

4.2 Iterative Resolution 6

4.3 Recursive Resolution 7

4.4 Caching and Negative Caching 8

4.5 Cache Poisoning: 2008-Style vs Modern Resolver 9

5 Experiments, Verification and Results **10**

5.1 Running the Module 10

5.2 Automated Verification 10

5.3 Observations 11

6 Conclusion **11**

AI Acknowledgement **11**

1 Introduction

People remember names; IP routes packets by address. The **Domain Name System** (**DNS**, RFC 1034 and RFC 1035) is the distributed database that maps one to the other. No single server could hold every name or answer every query, so the name space is a **tree** split into **zones**, each served by its own **authoritative** servers:

- **Root servers** know only who serves each top-level domain.
- **TLD servers** (`.edu`, `.com`, ...) know who serves each second-level domain.
- **Authoritative servers** hold the actual records of their zone.

A client does not walk this tree itself. It sends one query to a **resolver**, which either walks the tree (**iterative** resolution, following referrals) or passes the question to a server that does it on its behalf (**recursive** resolution). Every answer carries a **TTL**, and resolvers **cache** answers, including "no such name" (RFC 2308), which is what keeps DNS fast. DNS normally runs over **UDP port 53**, falling back to TCP when an answer does not fit in 512 bytes.

1.1 What Was Built

This module builds a complete DNS laboratory in pure Java 11 with no external libraries:

1. **RFC 1035 wire format:** encoding and decoding of all five message sections, name compression, and A, NS, CNAME, SOA, PTR, MX, TXT and AAAA records, with a byte-level dissector.
2. **A four-level hierarchy of real authoritative servers**, each on its own loopback address: a lab root, `edu.`, `myuniv.edu.` and `cse.myuniv.edu.` (which holds the Assignment 6 records), loaded from BIND-style zone files.
3. **Both resolution methods:** iterative and recursive, animated side by side message by message.
4. **A caching resolver** on port 53 (and 5300) that Windows `nslookup` can use, with TTL countdown, negative caching, bailiwick checks on glue, and detection of transparent DNS interception.
5. **UDP truncation** (TC=1) and **TCP fallback** for large answers.
6. **A cache-poisoning experiment:** 65,536 forged replies against a 2008-style resolver and against a modern one.
7. **A live dashboard** on `http://localhost:8084`.

Table 1: The lab name servers

Address	Server name	Zone
127.0.0.10	`a.root-servers.lab`	`.` (lab root): delegates `edu.` and the reverse zone
127.0.0.11	`ns.edu-servers.lab`	`edu.`: delegates `myuniv.edu.`
127.0.0.12	`ns1.myuniv.edu`	`myuniv.edu.`: delegates `cse.myuniv.edu.`
127.0.0.13	`dns.cse.myuniv.edu`	`cse.myuniv.edu.`: `www` = 192.168.151.4, `web` (CNAME), MX, ...
127.0.0.1:53 / :5300	resolver	Caching recursive resolver for clients

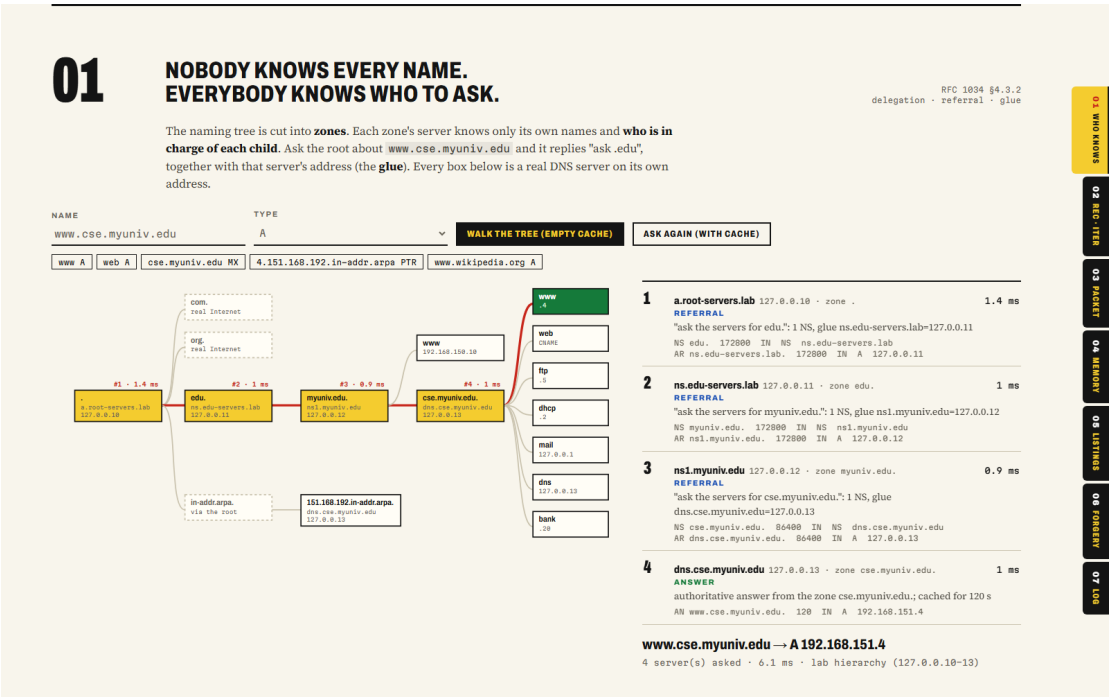


Figure 1: Walking the tree with an empty cache: root, `edu`, `myuniv`, `cse`.

2 The RFC 1035 Message Format

Queries and responses share one format: a fixed 12-byte header followed by four variable sections.

```
+----------------+-----------------------------------------------------+
|  ID (16 bits)  | QR | Opcode | AA | TC | RD | RA | Z | RCODE         |
+----------------+-----------------------------------------------------+
|  QDCOUNT       |  ANCOUNT       |  NSCOUNT       |  ARCOUNT          |
+----------------+-----------------------------------------------------+
|  Question:   QNAME  QTYPE  QCLASS                                     |
|  Answer:     resource records  (NAME TYPE CLASS TTL RDLENGTH RDATA)   |
|  Authority:  NS records of the zone that should be asked next         |
|  Additional: glue (A records of those name servers), ...              |
+-----------------------------------------------------------------------+
```

Figure 2: DNS message layout: 12-byte header and four sections.

Table 2: Header flags used in this lab

Flag	Meaning
QR	0 = query, 1 = response.
AA	Authoritative Answer: the responding server owns the zone.
TC	Truncated: the answer did not fit in a 512-byte UDP datagram; retry over TCP.
RD	Recursion Desired: set by the client to ask the server to find the final answer.
RA	Recursion Available: set by a server that is willing to recurse.
RCODE	0 NOERROR, 2 SERVFAIL, 3 NXDOMAIN, 5 REFUSED.

Name compression. A name is a sequence of length-prefixed labels ending in a zero

byte: `www.cse.myuniv.edu` is `03 www 03 cse 06 myuniv 03 edu 00`. Because the same suffix appears many times in a response, any suffix that occurred earlier is replaced by a 2-byte **pointer** whose top two bits are `11`: `C0 0C` means "the rest of this name is at offset 12", which is where the question name starts, right after the header.

```
void name(String n, boolean compress) {
    String rest = n;
    while (!rest.isEmpty()) {
        Integer p = compress ? ptr.get(rest.toLowerCase()) : null;
        if (p != null) { u16(0xC000 | p); return; }              // pointer to an earlier copy
        if (pos() < 0x3FFF) ptr.putIfAbsent(rest.toLowerCase(), pos());
        int dot = rest.indexOf('.');
        String label = dot < 0 ? rest : rest.substring(0, dot);
        byte[] b = label.getBytes(StandardCharsets.US_ASCII);
        if (b.length > 63) throw new IllegalArgumentException("label longer than 63 bytes: " + label);
        u8(b.length); out.write(b, 0, b.length);                 // length-prefixed label
        rest = dot < 0 ? "" : rest.substring(dot + 1);
    }
    u8(0);                                                       // root label
}
```

Listing 1: Encoding a name with compression pointers (`wire/Message.java`).

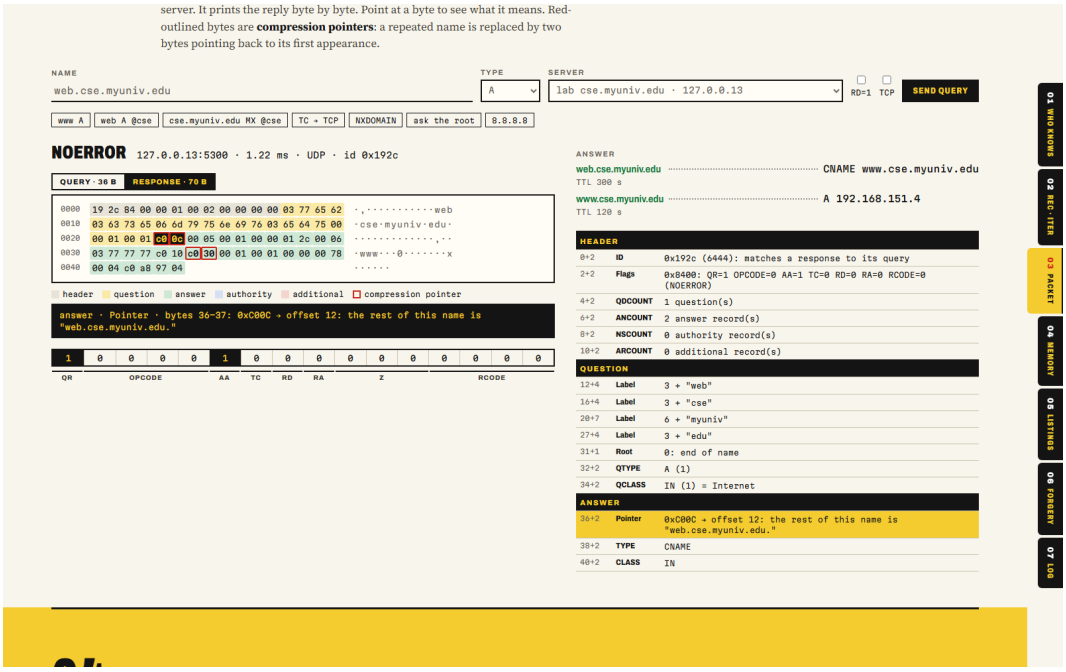


Figure 3: The byte-level dissector: every field of a real response, including the `C0 0C` compression pointer.

3 System Architecture

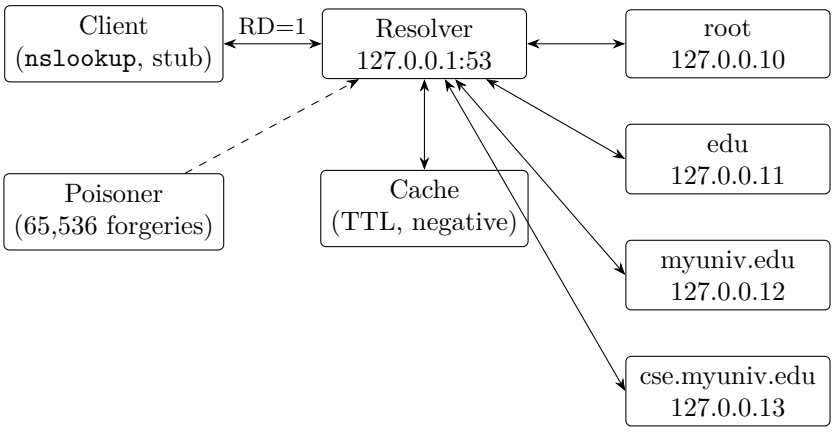


Figure 4: Components of the DNS module (iterative mode: the resolver asks every level itself).

Table 3: Source packages

Package	**Responsibility**
`wire/`	RFC 1035 encoder and decoder; records every field's byte span for the dissector.
`zone/`	BIND-style zone files: SOA, NS, A, CNAME, MX, PTR, TXT, AAAA.
`server/AuthServer`	Authoritative answers, referrals with glue, NXDOMAIN/NODATA with SOA, REFUSED, TC=1, and optional recursion.
`server/Listener`	UDP and TCP listeners (512-byte UDP limit, 2-byte length prefix on TCP).
`server/Resolver`, `Cache`	Iterative walk from root hints, bailiwick check, CNAME chasing, TTL and negative cache, secure upstream queries.
`attack/Poisoner`	The Kaminsky-style forgery race.
`client/DnsClient`	Stub client used by the tests and the dashboard.

4 Implementation and Workflows

4.1 Authoritative Servers

Each lab server answers from its own zones only. For a name in its zone it gives an authoritative answer (AA=1) and chases CNAMEs inside the zone, so `web.cse.myuniv.edu` returns the CNAME and the A record of `www`. For a name below a delegation it returns a **referral**: the NS records of the child zone in the Authority section and their A records as **glue** in the Additional section. A missing name gets NXDOMAIN and a missing type gets NODATA; both include the SOA so that resolvers can cache the negative answer. Names outside every zone get REFUSED. A UDP answer longer than 512 bytes is truncated with TC=1, and the client repeats the query over TCP.

4.2 Iterative Resolution

The resolver starts from the root hints, sends RD=0 queries, and follows each referral one level down. It accepts glue only if it lies inside the zone being delegated (the *bailiwick check*); otherwise a malicious server could inject addresses for unrelated names.

Table 4: Iterative lookup of `www.cse.myuniv.edu A` with an empty cache (measured)

Step	Server asked	Response
1	`a.root-servers.lab` (127.0.0.10)	REFERRAL: "ask the servers for `edu.`", glue `ns.edu-servers.lab` = 127.0.0.11
2	`ns.edu-servers.lab`	REFERRAL to `myuniv.edu.`, glue `ns1.myuniv.edu` = 127.0.0.12
3	`ns1.myuniv.edu`	REFERRAL to `cse.myuniv.edu.`, glue `dns.cse.myuniv.edu` = 127.0.0.13
4	`dns.cse.myuniv.edu`	ANSWER (AA=1): 192.168.151.4, TTL 120 s, now cached

A second query for the same name is answered from the cache with no server asked. A query for a sibling such as `mail.cse.myuniv.edu` starts directly at the cached `cse` servers: one hop instead of four.

4.3 Recursive Resolution

In recursive mode the resolver sends a single RD=1 query to the root. Each lab server that cannot answer asks the next one itself, waits, and relays the answer back up (RA=1). Both methods use ten messages for this lookup, but they spread the work very differently:

Table 5: The ten messages of each method, as replayed on the dashboard

#	Iterative	Recursive
1	PC → resolver, RD=1	PC → resolver, RD=1
2	resolver → root, RD=0	resolver → root, RD=1
3	root → resolver: referral to `edu`	root asks `edu` itself and waits
4	resolver → `edu`	`edu` asks `myuniv` and waits
5	`edu` → resolver: referral	`myuniv` asks `cse` and waits
6	resolver → `myuniv`	`cse` answers `myuniv`
7	`myuniv` → resolver: referral	`myuniv` relays to `edu`
8	resolver → `cse`	`edu` relays to root
9	`cse` → resolver: authoritative answer	root relays to the resolver
10	resolver → PC, RA=1	resolver → PC, RA=1

In recursive mode the root is busy for **8 of the 10 steps**, holding state for the whole chain below it; in iterative mode it is busy for only **2**. A root server that recursed for the whole Internet would be overwhelmed and would be an easy target for amplification attacks, so real roots refuse recursion. The dashboard has a switch that does the same: with recursion off, an RD=1 query to the root gets RA=0 and only a referral.

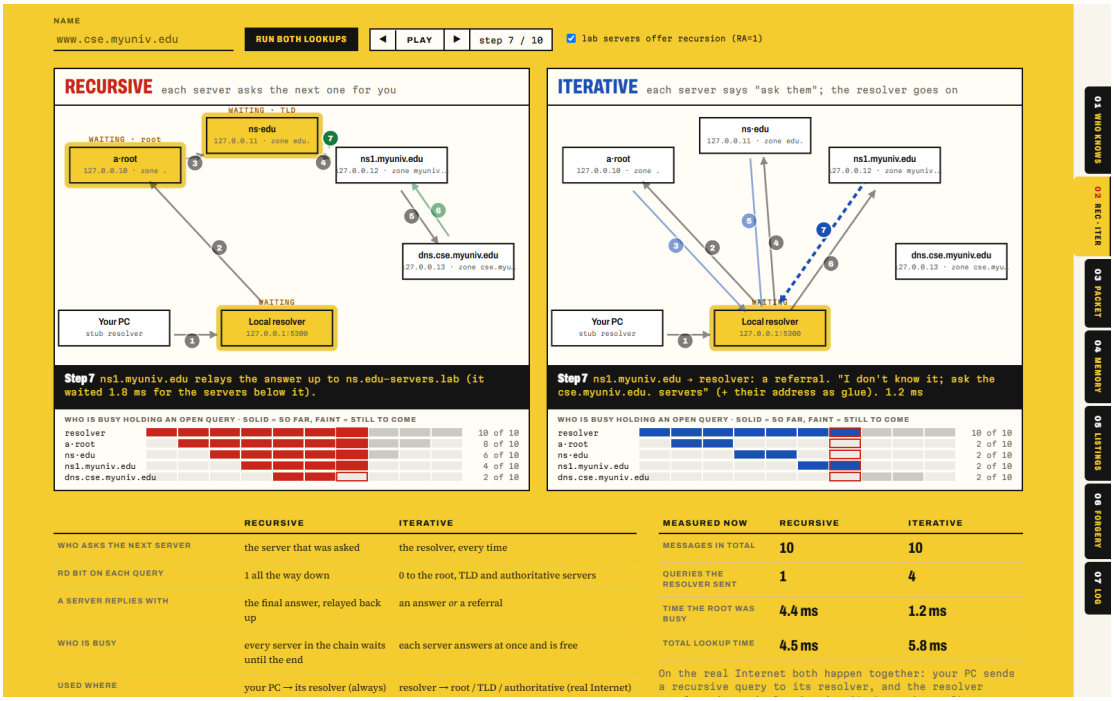


Figure 5: Recursive and iterative resolution side by side, with the "who is busy" strip.

4.4 Caching and Negative Caching

Every cached record counts its TTL down and is removed when it reaches zero. An NXDOMAIN or NODATA answer is cached for the SOA's minimum TTL (RFC 2308), so repeated queries for a name that does not exist do not reach the authoritative server again.

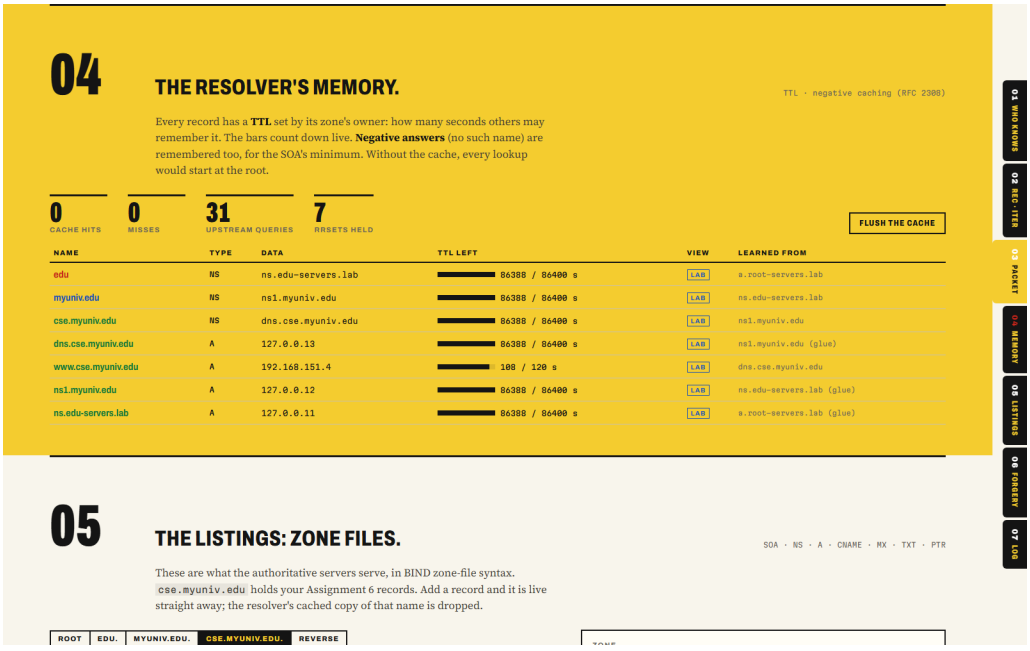


Figure 6: The resolver cache: TTL countdown bars and a negative (NXDOMAIN) entry.

4.5 Cache Poisoning: 2008-Style vs Modern Resolver

To poison a cache, an attacker must make the resolver accept a forged response before the real one arrives. A response is matched to its query by the 16-bit ID and the UDP port. In the experiment, the real answer for `bank.cse.myuniv.edu` is delayed by 1.5 s while the attacker sends **65,536 forged replies** claiming `bank` = 6.6.6.6, one for every possible ID.

- The **2008-style resolver** sends every upstream query from a fixed source port and checks nothing else. One forgery carries the right ID, and the false address is cached: **POISONED**.
- The **modern resolver** uses a fresh random source port and a random ID for every query, checks that the reply comes from the address and port it asked, and randomises the letter case of the query name ("0x20" encoding), requiring the reply to echo it exactly. Every forgery is rejected: **DEFENDED**.

```
try (DatagramSocket s = new DatagramSocket()) {        // a fresh random source port every query
    s.send(new DatagramPacket(out, out.length, to, port));
    while (System.currentTimeMillis() < deadline) {
        DatagramPacket p = new DatagramPacket(buf, buf.length);
        try { s.receive(p); } catch (SocketTimeoutException e) { continue; }
        if (!p.getAddress().equals(to) || p.getPort() != port) { rejected.incrementAndGet(); continue; }  // wrong source
        if (p.getLength() < 12 || ((buf[0] & 0xFF) << 8 | buf[1] & 0xFF) != id) { rejected.incrementAndGet(); continue; }  // wrong ID
        Message m = Message.decode(buf, p.getLength());
        if (m.qd.isEmpty() || !m.qd.get(0).name.equals(qname) || m.qd.get(0).type != type) {
            rejected.incrementAndGet(); continue;          // 0x20: the name must come back in exactly our random case
        }
        return exchange(m, p);
    }
}
```

Listing 2: Accepting an upstream reply safely (`server/Resolver.java`).

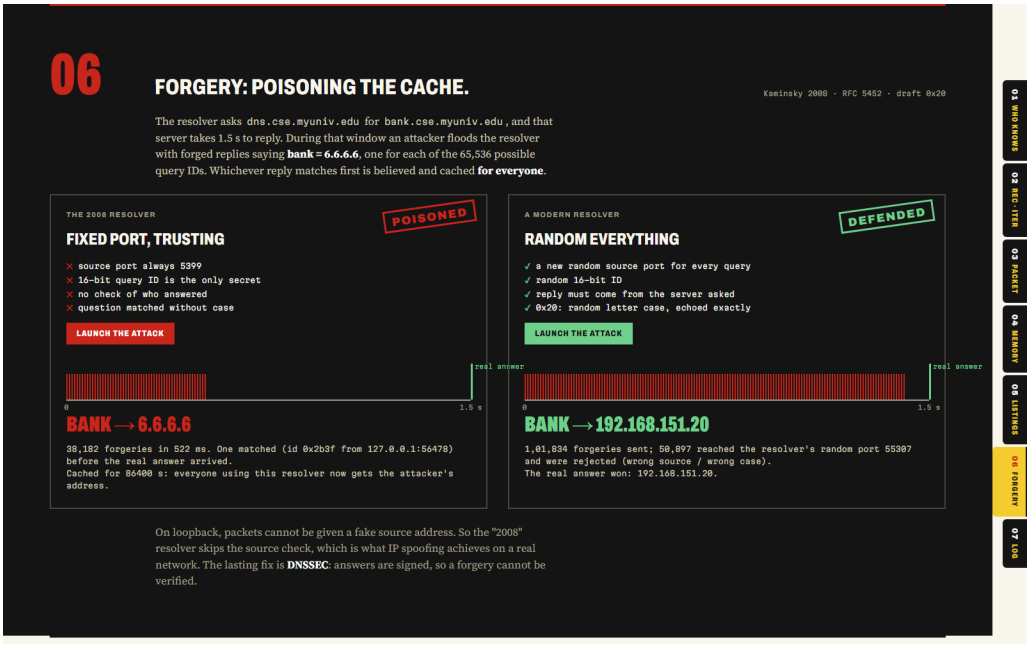


Figure 7: The poisoning race: the 2008-style resolver is poisoned, the modern resolver rejects every forgery.

5 Experiments, Verification and Results

5.1 Running the Module

```
cd DNS
run.bat                  # four lab servers, resolver and dashboard (http://localhost:8084)
run.bat --verify         # 31 automated checks
python interop_test.py   # Windows nslookup and raw packets against our servers (13 checks)

nslookup www.cse.myuniv.edu 127.0.0.1                 # through our resolver on port 53
nslookup -type=MX cse.myuniv.edu 127.0.0.1
nslookup -norecurse www.cse.myuniv.edu 127.0.0.10     # iterative: the root only refers
nslookup 192.168.151.4 127.0.0.1                      # reverse lookup (PTR)
```

Listing 3: Starting the hierarchy and querying it with Windows nslookup.

5.2 Automated Verification

All **31 checks** of `Verify.java` pass, and all 13 interoperation checks with Windows `nslookup` and hand-built packets pass.

Table 6: Selected verification checks (all passed)

Encode → decode round trip keeps ID, flags and records; the header is exactly 12 bytes
Name compression: repeated names become 2-byte pointers (0xC0…)
`www.cse.myuniv.edu A` = 192.168.151.4 with AA=1; CNAME `web` → `www` with the A record added
NXDOMAIN and NODATA carry the SOA; a name outside every zone gets REFUSED
Root: referral to `edu.` with glue (AA=0, no answer)
Answer > 512 bytes: the UDP reply has TC=1 and the client retries over TCP
MX with its A record as additional; reverse lookup 192.168.151.4 → `www.cse.myuniv.edu` (PTR)
Iterative walk root → edu → myuniv → cse: three referrals, then the answer
Second lookup straight from the cache; the cached record carries the remaining TTL; a sibling name needs one hop
NXDOMAIN, then a cached negative answer
One RD=1 query to the root returns the final answer; the root stays busy for the whole chain
Recursion switched off (like real roots): RA=0 and only a referral
Vulnerable resolver: forged answer cached. Secure resolver: not poisoned

5.3 Observations

1. **Caching dominates.** The first iterative lookup took about 58 ms over four servers; the repeated lookup needed no server at all, and a sibling name needed one.
2. **Recursion moves the load up the tree.** With the same ten messages, recursive mode kept the root busy for the whole chain, which is why real root and TLD servers only refer.
3. **Randomness is the defence against forgery.** A fixed source port leaves only 65,536 IDs to guess, which the attacker covers in one burst; a random port, random ID, source check and 0x20 case together make blind forgery impractical.
4. **Our resolver is a real one.** Windows `nslookup` resolves lab names through it on port 53, including reverse lookups and MX queries.

6 Conclusion

1. **Hierarchy plus delegation lets DNS scale.** Each server knows only its own zone and where to send the client next; referrals with glue connect the levels.
2. **Iterative and recursive resolution differ in who does the work.** Measuring both on the same four servers showed that recursion keeps the upper levels busy, which explains why the public root refuses it.
3. **Caching makes DNS fast, and must be protected.** TTLs and negative caching remove most queries, but a cache that accepts a forged answer serves it to every client; source-port and ID randomisation with 0x20 case checks defeated all 65,536 forgeries in the experiment.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.

- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.